Idempotent, Monotonic Webhook Ingestion Engine with Crash-Safe Local State Durability

Candidate Assessment Report · Software Engineering Practice

October 18, 2026

Abstract — At-least-once webhook delivery introduces duplicate packets, out-of-order versions, status collisions, and mid-write crash risks. This paper presents a zero-dependency Python 3.10+ ingestion engine that enforces tenant isolation, deduplicates payloads via canonical matching, guarantees monotonic version progression, and persists state via atomic POSIX `os.replace`. We formalize six Architectural Decision Records (ADRs)—highlighting immediate key reservation on version conflicts (Row 9 replay hazard) and strict boolean type exclusion—and verify the system across three fixture stages and 18 unit tests with zero byte mutations on corrupt state.

1 Problem Formulation

Upstream webhook publishers use timeout-driven retries with exponential backoff, exposing receivers to four failure modes (Figure 1):

1. **Duplicate Ingestion:** Identical payloads redelivered after network timeouts.
2. **Stale Hazards:** Out-of-order older versions arriving after newer state transitions.
3. **Version Conflicts:** Contradictory statuses arriving at the same version number.
4. **Truncated Writes:** Process termination corrupting in-place file writes.

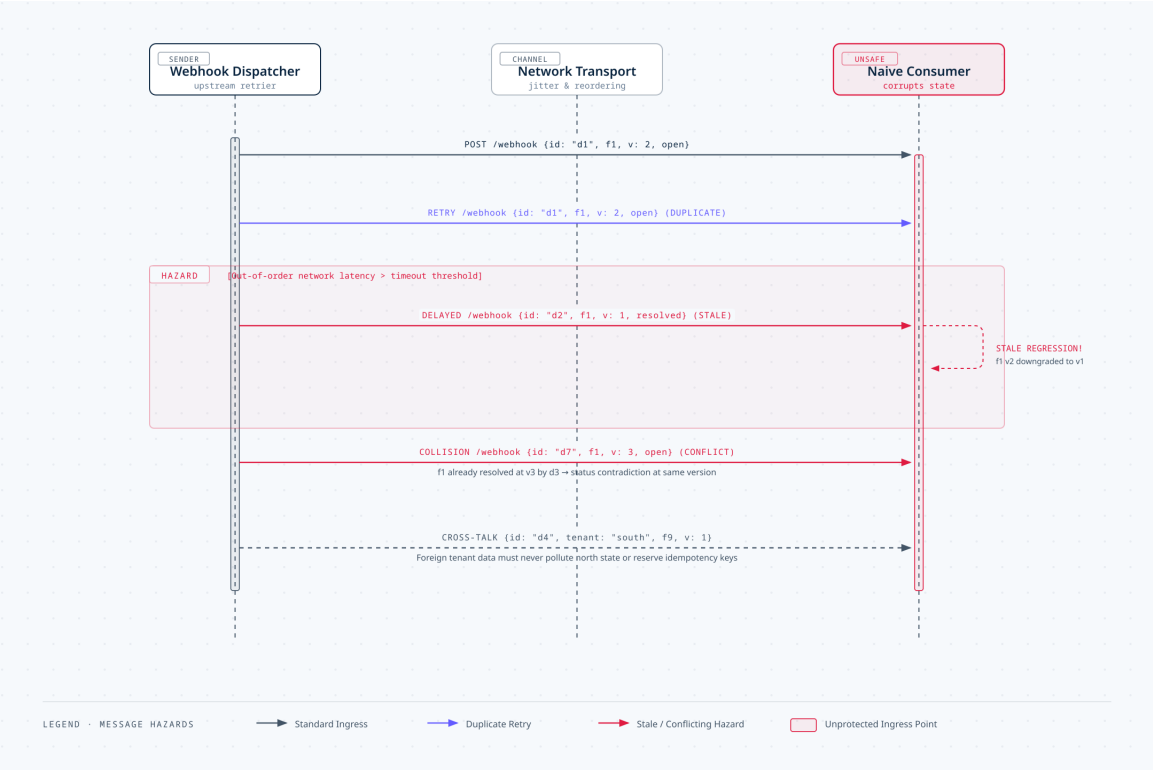


Figure 1: Sequence analysis of webhook redeliveries, latency hazards, and state regressions.

Our objective is a self-contained CLI that ingests synthetic batches into a local JSON state file while preserving idempotency and monotonic invariants across process runs.

2 System Invariants and Formal Contract

State is partitioned into the *Delivery Registry* (transport identity) and the *Finding Registry* (domain state). A normalized delivery row is a tuple:

$$D = \langle \mathrm{id}, \tau, \phi, v, \sigma \rangle, \quad \mathrm{id}, \tau, \phi \in \Sigma^{+},\ v \in \mathbb{Z}^{+},\ \sigma \in \{\mathrm{open}, \mathrm{resolved}\} \tag{1}$$

- **Invariant 1 (Tenant & Schema Gate):** Rows failing schema validation or matching $\tau \neq \tau_{\text{CLI}}$ return `rejected` without reserving an idempotency key or leaking foreign payloads.
- **Invariant 2 (Delivery Identity Gate):** For a known id with stored payload $P_{\text{stored}} = \langle \tau, \phi, v, \sigma \rangle$:

$$\text{Outcome}(\text{id}) = \begin{cases} \text{duplicate}, & P_{\text{in}} = P_{\text{stored}} \\ \text{rejected (delivery conflict)}, & P_{\text{in}} \neq P_{\text{stored}} \end{cases} \tag{2}$$

  Neither outcome mutates findings.
- **Invariant 3 (Immediate Key Reservation):** Every valid, unseen id is recorded in state`["deliveries"]` *before* version comparison, even if its version is stale, redundant, or conflicting.
- **Invariant 4 (Monotonic Progression):** Given finding state $(v_{\text{curr}}, \sigma_{\text{curr}})$:

$$\text{Outcome}(v_{\text{in}}, \sigma_{\text{in}}) = \begin{cases} \text{applied}, & v_{\text{in}} > v_{\text{curr}} \implies \text{update } (v_{\text{in}}, \sigma_{\text{in}}) \\ \text{stale}, & v_{\text{in}} < v_{\text{curr}} \implies \text{no-op} \\ \text{redundant}, & v_{\text{in}} = v_{\text{curr}} \wedge \sigma_{\text{in}} = \sigma_{\text{curr}} \implies \text{no-op} \\ \text{rejected}, & v_{\text{in}} = v_{\text{curr}} \wedge \sigma_{\text{in}} \neq \sigma_{\text{curr}} \implies \text{no-op} \end{cases} \tag{3}$$

- **Invariant 5 (Atomic Durability):** State is committed only after a valid batch via a sibling temporary file and POSIX `os.replace`. Corrupt or foreign state files exit 2 with zero byte changes.

3 Architecture and Decision Records (ADR)

Figure 2 depicts the two-tier ingestion pipeline, and Table 1 summarizes the six core ADRs.

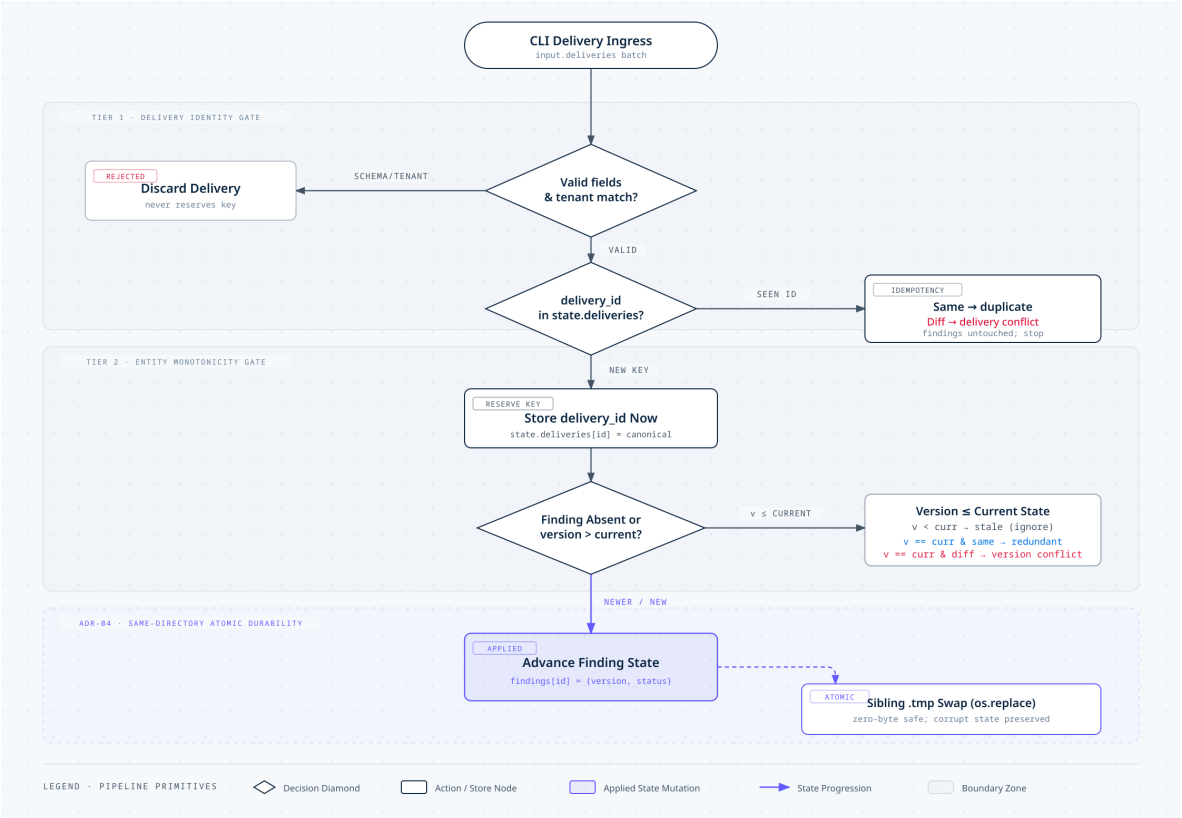


Figure 2: Two-tier ingestion architecture decoupling transport deduplication from domain monotonicity, backed by sibling-file atomic POSIX persistence (ADR-04).

3.1 Row 9 Key Reservation Trap (ADR-03) & Crash Atomicity (ADR-04)

Row 9 Trap: In `primary.json`, Row 9 ($\langle$d7, north, f1, 3, open$\rangle$) collides with finding `f1` at $\{v = 3, \text{resolved}\}$ (set by Row 5, `d3`), yielding `rejected` (`version conflict`). Discarding `d7` causes Run 2 replay to re-evaluate `d7` as a conflict (`duplicate: 5, rejected: 4`). Reserving `d7` immediately after schema validation converts Run 2 into a `duplicate`, matching the required `duplicate: 6, rejected: 3`.

POSIX Atomicity: Opening a file with `"w"` invokes `truncate(2)` immediately, risking data loss on crash. Staging in `/tmp` risks cross-filesystem (`EXDEV`) non-atomic copies. Writing to `.filename.tmp` in the target parent directory guarantees a single-inode atomic swap via `os.replace`.

Table 1: Architectural Decision Record (ADR) Summary Matrix

ID	Area	Chosen Architecture	Alternative Rejected	Rationale
ADR-01	Ingestion	Sequential loop with live mutation	Pre-filtering (`seen = set()`)	Preserves 1-based row indexing and intra-batch transitions.
ADR-02	Pipeline	Two-tier: Delivery Gate → Entity Gate	Single unified conditional	Separates transport replay from domain version rules.
ADR-03	Key Storage	Reserve valid ID before version check	Lazy reservation on `applied`	Ensures version conflicts (Row 9) deduplicate on replay.
ADR-04	Durability	Sibling temp file + `os.replace`	In-place `open(path, "w")`	Prevents 0-byte truncation on crash and cross-mount copies.
ADR-05	Integrity	Fail-fast (exit 2) on corrupt state	Silent fallback to empty state	Guarantees forensic byte preservation on invalid state files.
ADR-06	Type Safety	`type(v) is int` (excludes `bool`)	`isinstance(v, int)`	Rejects `"version": true` since `bool` subclasses `int`.

4 Human-AI Collaborative Verification

Engagement began with a zero-code conceptual prompt (`ai_use/01-Explanation.md`) to establish invariants and test boundaries prior to coding:

> **Developer Prompt** (`ai_use/01-Explanation.md`)
>
> *"Explain the problem to me clearly before I start solving it. Break down the requirements, explain what is expected from me, provide practical examples and edge cases, and suggest how I should approach the problem step by step. Do not give me the solution, write code, or implement anything."*

All AI outputs were verified against fixture traces:

- **Accepted (ADR-04):** Adopted sibling temporary file staging + `Path.replace()` with `unlink()` cleanup.
- **Corrected Hallucination (ADR-03):** AI skipped saving non-applied keys; corrected via Row 9 replay trace to reserve keys immediately upon schema validation.
- **Rejected (ADR-01):** Rejected batch pre-deduplication (`seen = set()`) because it breaks 1-based indexing and hides delivery conflicts.
- **Corrected Type Check (ADR-06):** Replaced AI's `isinstance(v, int)` with `type(v) is int` to block `bool` coercion.

5 Core Implementation Artifacts

Listings 1–3 present the production implementation in `app.py`.

```python
def validate_delivery_row(
    row: typing.Any, expected_tenant: str
) -> tuple[bool, dict[str, typing.Any] | None, str, str | None]:
    if not isinstance(row, dict):
        return False, None, "", "row must be an object"

    raw_delivery_id = row.get("delivery_id")
    raw_tenant = row.get("tenant")
    raw_finding_id = row.get("finding_id")
    version = row.get("version")
    raw_status = row.get("status")

    delivery_id = str(raw_delivery_id).strip() if isinstance(raw_delivery_id, str) else ""

    if not isinstance(raw_delivery_id, str) or not raw_delivery_id.strip():
        return False, None, delivery_id, "delivery_id must be a non-empty string"
    if not isinstance(raw_tenant, str) or not raw_tenant.strip():
        return False, None, delivery_id, "tenant must be a non-empty string"
    if not isinstance(raw_finding_id, str) or not raw_finding_id.strip():
        return False, None, delivery_id, "finding_id must be a non-empty string"

    tenant = raw_tenant.strip()
    if tenant != expected_tenant:
        return False, None, delivery_id, f"foreign tenant: {tenant}"

    # Strict type identity check excluding bool
    if type(version) is not int or isinstance(version, bool) or version <= 0:
        return False, None, delivery_id, "version must be a positive integer"

```

```
    if not isinstance(raw_status, str):
        return False, None, delivery_id, "status must be a string"
    status = raw_status.strip()
    if status not in VALID_STATUSES:
        return False, None, delivery_id, f"status must be 'open' or 'resolved', got '{status}'"

    canonical = {
        "tenant": tenant,
        "finding_id": raw_finding_id.strip(),
        "version": version,
        "status": status,
    }
    return True, canonical, delivery_id, None
```

Listing 1: Row validation with strict boolean exclusion (`app.py`).

```
def save_state(state: dict[str, typing.Any], state_path: pathlib.Path) -> None:
    """Atomically save state JSON file via a temporary file in the same directory."""
    target = state_path.resolve()
    target.parent.mkdir(parents=True, exist_ok=True)
    temp_path = target.with_name(f".{target.name}.tmp")
    try:
        content = json.dumps(state, indent=2, sort_keys=True) + "\n"
        temp_path.write_text(content, encoding="utf-8")
        temp_path.replace(target)
    except Exception as error:
        if temp_path.exists():
            try:
                temp_path.unlink()
            except OSError:
                pass
        raise ValueError(f"failed to save state: {error}") from error
```

Listing 2: Crash-safe sibling temporary file serialization (`app.py`).

```
for row_idx, row in enumerate(deliveries, start=1):
    is_valid, canonical, delivery_id, reason = validate_delivery_row(row, arguments.tenant)
    if not is_valid:
        results.append({"row": row_idx, "delivery_id": delivery_id, "outcome": "rejected", "
            reason": reason})
        counts["rejected"] += 1
        continue

    # Tier 1: Delivery Identity Gate
    if delivery_id in state["deliveries"]:
        stored = state["deliveries"][delivery_id]
        if stored == canonical:
            outcome, reason = "duplicate", "duplicate delivery"
            counts["duplicate"] += 1
        else:
            outcome, reason = "rejected", "delivery conflict"
            counts["rejected"] += 1
        results.append({"row": row_idx, "delivery_id": delivery_id, "outcome": outcome, "reason"
            : reason})
        continue

    # Step 3: Immediate Key Reservation
    state["deliveries"][delivery_id] = canonical

    # Tier 2: Entity Monotonicity Gate
    finding_id = canonical["finding_id"]
    version, status = canonical["version"], canonical["status"]
    current = state["findings"].get(finding_id)

    if current is None or version > current["version"]:
        state["findings"][finding_id] = {"version": version, "status": status}
        outcome, reason = "applied", "finding updated"
        counts["applied"] += 1
    elif version < current["version"]:
        outcome, reason = "stale", "stale version"
        counts["stale"] += 1
    else:  # version == current["version"]
        if status == current["status"]:
            outcome, reason = "redundant", "redundant status"
            counts["redundant"] += 1
```

```
        else:
            outcome, reason = "rejected", "version conflict"
            counts["rejected"] += 1

    results.append({"row": row_idx, "delivery_id": delivery_id, "outcome": outcome, "reason":
        reason})
```

Listing 3: Two-tier sequential ingestion loop (`app.py`).

6 Empirical Verification and Edge Case Trace

Acceptance Runs:

1. **Run 1 (Fresh Primary):** `applied: 3, duplicate: 1, stale: 1, redundant: 0, rejected: 4` (Exit 0).
2. **Run 2 (Primary Replay):** `applied: 0, duplicate: 6, stale: 0, redundant: 0, rejected: 3` (Exit 0; rows 1, 2, 4, 5, 8, 9 deduplicate).
3. **Run 3 (Secondary):** `applied: 2, duplicate: 0, stale: 0, redundant: 1, rejected: 0` (Exit 0; `f1`: $\{v = 4, \text{open}\}$, `f2`: $\{v = 2, \text{resolved}\}$).

Table 2: Automated 18-Test Regression Suite (`tests/app_test.py`)

Test Name	Verified Invariant	St.	Test Name	Verified Invariant	St.
`whenHelpRequested`	Usage stdout, exit 0	P	`whenVersionZero`	$v \leq 0$ rejected	P
`whenInputMissing`	Stderr diagnostic, exit 2	P	`whenVersionBoolean`	Bool exclusion guard	P
`whenJsonMalformed`	Corrupt JSON, exit 2	P	`whenMissingField`	Missing key rejected	P
`whenPrimaryRun`	Fresh counts (3/1/1/0/4)	P	`whenMismatch`	Tenant mismatch, exit 2	P
`whenPrimaryReplayed`	Replay counts (0/6/0/0/3)	P	`whenStatusCase`	`"OPEN"` rejected	P
`whenSecondaryRun`	Counts (2/0/0/1/0)	P	`whenDeliveryIdInt`	Non-str ID rejected	P
`whenStateMalformed`	0-byte mutation, exit 2	P	`whenEmptyDeliveries`	Empty batch succeeds	P
`whenWhitespace`	Trims canonical strings	P	`whenStateTenantMismatch`	Foreign state preserved	P
`whenForeignTenant`	No key reserved	P	`whenVersionConflict`	Row 9 replay proof	P

Table 3: Row-by-Row Execution Trace of `inputs/primary.json`

Row	ID	Payload	Evaluation Step	Run 1	Key Stored?	State Mutation
1	d1	f1, $v = 2$, open	Finding absent $\implies$ init	applied	**Yes**	f1: $\{v = 2, \text{open}\}$
2	d1	f1, $v = 2$, open	Seen d1 $\wedge$ identical payload	duplicate	Stored	None
3	d1	f1, $v = 2$, resolved	Seen d1 $\wedge$ conflicting status	rejected	Stored	None
4	d2	f1, $v = 1$, resolved	New ID; $v = 1 < 2$ (stale)	stale	**Yes**	None
5	d3	f1, $v = 3$, resolved	New ID; $v = 3 > 2$ (advance)	applied	**Yes**	f1: $\{v = 3, \text{resolved}\}$
6	d4	f9, $v = 1$, open (south)	Foreign tenant validation fail	rejected	**No**	None
7	d5	f2, $v = 0$, open	Version $0 \leq 0$ validation fail	rejected	**No**	None
8	d6	f2, $v = 1$, open	Finding absent $\implies$ init	applied	**Yes**	f2: $\{v = 1, \text{open}\}$
9	d7	f1, $v = 3$, open	New ID; $v = 3 \wedge \text{open} \neq \text{resolved}$	rejected	**Yes**	None

7 Scope Boundaries and Conclusion

Scope & Next Steps: Process-level file locks (`fcntl.flock`) and SQLite backends were intentionally deferred per the sequential JSON CLI specification. For high-throughput production streams, future iterations will introduce streaming JSON parsers (`ijson`) and SQLite WAL-mode storage.

Conclusion: Decoupling delivery deduplication from entity monotonicity—backed by immediate key reservation and POSIX sibling-file replacement—guarantees deterministic state consistency across process lifecycles.

Appendix: Persisted State After Run 3


```
{
  "deliveries": {
    "d1": {"finding_id": "f1", "status": "open", "tenant": "north", "version": 2},
    "d2": {"finding_id": "f1", "status": "resolved", "tenant": "north", "version": 1},
    "d3": {"finding_id": "f1", "status": "resolved", "tenant": "north", "version": 3},
    "d6": {"finding_id": "f2", "status": "open", "tenant": "north", "version": 1},
    "d7": {"finding_id": "f1", "status": "open", "tenant": "north", "version": 3},
    "s1": {"finding_id": "f1", "status": "open", "tenant": "north", "version": 4},
```

```
    "s2": {"finding_id": "f2", "status": "open", "tenant": "north", "version": 1},
    "s3": {"finding_id": "f2", "status": "resolved", "tenant": "north", "version": 2}
  },
  "findings": {
    "f1": {"status": "open", "version": 4},
    "f2": {"status": "resolved", "version": 2}
  },
  "tenant": "north"
}
```

Listing 4: Final serialized state (`scratch-state.json`) after Run 3.